

Студент: Чернышева Елизавета Сергеевна  
Курс: 4  
Группа: Z3344  
2025 г.

# Лабораторная работа №1

## Алгоритмы сортировки

### 1 Цель работы

Познакомить студента с основами анализа алгоритмов на примере операций сортировки.

### 2 Задачи

#### 1. [СОРТ 1] Алгоритм сортировки 1

- 1.1 Сформулировать задачу, для которой требуется применение алгоритма сортировки;
- 1.2 Выбрать алгоритм сортировки. Доказать, что выбранный алгоритм является наилучшим выбором. Привести расчет ёмкостных и вычислительных затрат;
- 1.3 Реализовать алгоритм на языке C++;

#### 2. [СОРТ 2] Алгоритм сортировки 2

- 2.1 Сформулировать задачу, для которой требуется применение алгоритма сортировки. Алгоритм сортировки 2 должен отличаться от алгоритма сортировки 1. Асимптотическая сложность алгоритма 2 должна быть больше, чем у алгоритма 1;
- 2.2 Выбрать алгоритм сортировки. Доказать, что выбранный алгоритм является наилучшим выбором. Привести расчет ёмкостных и вычислительных затрат;
- 2.3 Реализовать алгоритм на языке C++.

#### 3. [SAVE] Всё, что было сделано в шагах 1-2, сохранить в репозиторий (+ отчет по данной ЛР в папку doc).

### 3 Решение

#### 1. [СОРТ 1] Алгоритм сортировки 1

##### Формулировка задачи:

На школьном турнире по программированию участникам выдавались идентификационные карточки с номерами. Номера карточек находятся в диапазоне от 0 до 20. После турнира

карточки собрали в кучу в произвольном порядке. Необходимо быстро отсортировать номера карточек по возрастанию для внесения в электронную базу данных.

### **Входные данные:**

- Первое число  $n$  ( $1 \leq n \leq 1000$ ) — количество карточек.
- Затем вводится  $n$  целых чисел  $k$ , каждое в диапазоне  $[0, 20]$  — номера карточек.

### **Входные данные:**

Отсортированный по возрастанию список номеров карточек.

Для такой задачи, при малых значениях  $k$ , лучше всего подойдет сортировка подсчетом. При этом алгоритме сортировки используется диапазон чисел сортируемого массива для подсчета совпадающих элементов, после чего числа выводятся в необходимом порядке фиксированное алгоритмом число раз - сколько каждое из них встретилось во входных данных.

Сложность такого алгоритма  $O(n + k)$ , что при малых  $k$  значительно быстрее, чем  $O(n^2)$  или  $O(n \log n)$ .

### **Реализация алгоритма сортировки подсчетом:**

- Создается массив `count` размером 21 (по количеству возможных чисел от 0 до 20), все элементы инициализируются нулями. Индекс массива соответствует возможному числу, а значение — количеству его вхождений.
- Считываем каждое число из ввода. Для числа `num` увеличиваем значение `count[num]` на 1.
- Проходим по массиву `count` от начала до конца (от наименьшего числа 0 до наибольшего 20). Для каждого числа `num` выводим его `count[num]` раз.

### **Вычислительная сложность алгоритма (затраты по времени):**

В первом цикле мы считаем сколько раз каждое число встретилось во входном массиве, то есть пробегаемся по каждому числу - это  $O(n)$  и читаем его - это  $O(1)$ , итого в первом цикле получаем  $O(n)$ . Во втором вложенном цикле мы восстанавливаем отсортированный массив - во внешнем  $k$  операций, во внутреннем  $n$ . Значит общая вычислительная сложность  $O(n + k)$ .

### **Емкостная сложность алгоритма (затраты по памяти):**

Мы создаем вектор, в котором храним то, сколько раз каждый номер встретился в массиве, и этот вектор зависит только от значений, которые может принимать  $k$ , и не зависит от того, сколько всего будет карточек с этими номерами, то есть не зависит от  $n$ . Получается константное количество памяти -  $O(1)$ .

## **2. [СОРТ 2] Алгоритм сортировки 2**

### Формулировка задачи:

Отдел кадров компании хранит данные о сотрудниках, содержащих имя и зарплату. Необходимо отсортировать сотрудников по убыванию зарплаты. Если зарплаты одинаковые, отсортировать по имени в алфавитном порядке.

В такой задаче лучше подойдет уже быстрая сортировка. Во-первых, зарплаты могут являться нецелочисленными, поэтому возникает проблема при применении сортировки подсчетом. Во-вторых, зарплаты могут быть в неограниченном диапазоне неотрицательных чисел, а это значит, что число  $k$  из первой задачи может быть очень большим и сильно превосходить  $n$ , поэтому сложность алгоритма может превосходить  $O(n^2)$  или  $O(n \log n)$  (например  $2^n$ ). Также быстрая сортировка позволяет работать с такими составными структурами, где можно сортировать не только числа, но и имена.

### Реализация алгоритма быстрой сортировки:

- Выбираем опорный элемент `pivot` из середины.
- Ставим указатели на концах массивов - `left` и `right`.
- В цикле `while` ищем элементы-"нарушители" с двух сторон от опорного элемента, если находим, меняем их местами и сдвигаем указатели.
- После выхода из цикла - рекурсивная часть, снова прогоняем разделенные части массива через цикл.

### Вычислительная сложность алгоритма (затраты по времени):

Т.к. мы используем рекурсию, мы каждый раз делим пополам и спускаемся на уровень ниже, пока не дойдем до последнего уровня, где останутся только пары  $\Rightarrow$  всего уровней  $\log_2(n)$ . На каждом уровне мы прогоняем цикл `while`, на котором обрабатываем все  $n$  элементов. Получается вычислительная сложность  $O(n \log n)$  в лучшем случае.

Худший случай, когда массив уже отсортирован и мы выбираем в качестве `pivot` первый или последний элемент массива. Тогда вычислительная сложность становится  $O(n^2)$ . Выбирая `pivot` из середины массива, мы уменьшаем вероятность наступления худшего случая.

### Емкостная сложность алгоритма (затраты по памяти):

В лучшем случае глубина рекурсии  $\log_2(n)$ , на каждом уровне используется константное количество памяти для вызова функции сортировки (переменные `start`, `end`, `left`, `right`, `mid`, `pivot`), то есть  $O(1)$ , значит всего емкостная сложность в лучшем случае составляет  $O(\log n)$ .

В худшем случае глубина рекурсии на каждой итерации будет  $n$ ,  $(n-1)$ ,  $(n-2)$ , ...,  $1$ , на каждый вызов функции так же уходит константное количество памяти, итого получаем емкостную сложность  $O(n)$ .

3. [SAVE] Всё, что было сделано в шагах 1-2, сохранить в репозиторий (+ отчет по данной ЛР в папку `doc`).